BBO Capstone: Overview of Our Approach

[Your name] | Imperial College Professional Certificate in ML & AI

1. Overview of our BBO approach

What we are trying to achieve

- Find the inputs that give the **highest output** for each of eight unknown ("black-box") functions. They have between 2 and 8 inputs each, all between 0 and 1.
- We cannot see inside a function: there is no formula and no gradient. We only get one number back for each point we try.
- We may submit just **one new point per function per week**, so every query has to count. Success means the best value found for each function.

Our overall strategy

Each week we repeat the same five-step loop:

1. **Model.** A Gaussian process (GP) is fitted to all results so far. It predicts the output at any untried point and says how unsure it is.
2. **Choose.** We pick the point with the highest "predicted value plus a bonus for uncertainty" (upper confidence bound, UCB). A small bonus exploits what looks good; a larger one explores unknown areas. Each week we compare two bonus weights (1.0 and 1.96).
3. **Check.** We reject repeats of earlier points and test whether the model's uncertainty can be trusted (leave-one-out and forward-chaining calibration).
4. **Submit.** One point per function.
5. **Learn.** We add the result to the data and refit the model.

How the process fits together

- **Tuned per function.** Kernel type, length-scale bounds and noise level are chosen separately for each function by log marginal likelihood.
- **Function-specific fixes.** Function 5 uses a log transform (its outputs span about 48,000 times). Function 6 allows extra noise, because the same input returned two different outputs.
- **Trust where earned.** For Functions 5 and 8 the model proved reliable, so we added TuRBO, a local search inside a trust region around the best point. For the others the model was poorly calibrated, so we kept to cautious steps near the best known points.
- **Over time.** The strategy moved from broad exploration early on to focused exploitation, with the final queries aimed at small refinements of each function's best point.

2. How our strategy has evolved

Key changes since the early rounds

- **Early rounds:** one default GP (RBF kernel) with UCB at 1.96 for every function, plus a manual pick for Function 1.
- **Per-function settings:** after the length-scales collapsed for Functions 1 and 2, kernel type, length-scale bounds and noise level were chosen separately for each function by likelihood.
- **Safeguards:** a duplicate detector (Function 5 had been submitted three times at the same point) and calibration checks on the model's uncertainty.
- **Targeted fixes:** a log transform for Function 5 and a noise-aware setting for Function 6.
- **Alternatives tested:** linear, SVM and neural-network models, and a hybrid strategy for Function 1. The GP stayed the main model.
- **TuRBO** (a local trust-region search) was added for Functions 5 and 8 in week 11, and the focus moved from broad exploration to exploiting around the best points.

What influenced these changes

- **Data trends:** good results clustered (the top results of Functions 3 to 7 sit close together), Function 5's outputs span about 48,000 times, and Function 8 plateaued between 9.74 and 9.91 until TuRBO reached 9.998.
- **Model performance:** kernel collapse on Functions 1 and 2, a Function 5 prediction missed by 7.44 standard deviations, and poor calibration on Functions 4, 6 and 7.
- **Experiments:** Function 6 returned two different outputs for one input, which showed it is noisy. Week 11's TuRBO results beat their predictions.
- **Course material and intuition:** the TuRBO tutorial and the NeurIPS 2020 challenge papers shaped what we tried. Intuition played a small role, mainly early manual picks, and was later checked against the numbers.

Principles that now guide each query

- Trust a model only as far as its uncertainty proves honest.
- Do not waste a query: avoid repeats, except deliberately for the noisy Function 6.
- Exploit where results cluster; explore where the model has no signal.
- Explore early and exploit late. The final query is a small refinement of each function's best point.
- The score is the best value found, so a poor query costs little. Take small risks before the final round.
- Test a change (for example with forward-chaining) before adopting it.